CSE 4137

Machine Learning
Module-05
Random Forest



Md. Shahid Uz Zaman
Adjunct Faculty
Dept of CSE, BAUET
szaman22.ruet@gmail.com

Topics: KNN

Week-08

- Introduction
- Implementation
- Python Example

Ensemble Learning

Week-06, c-3

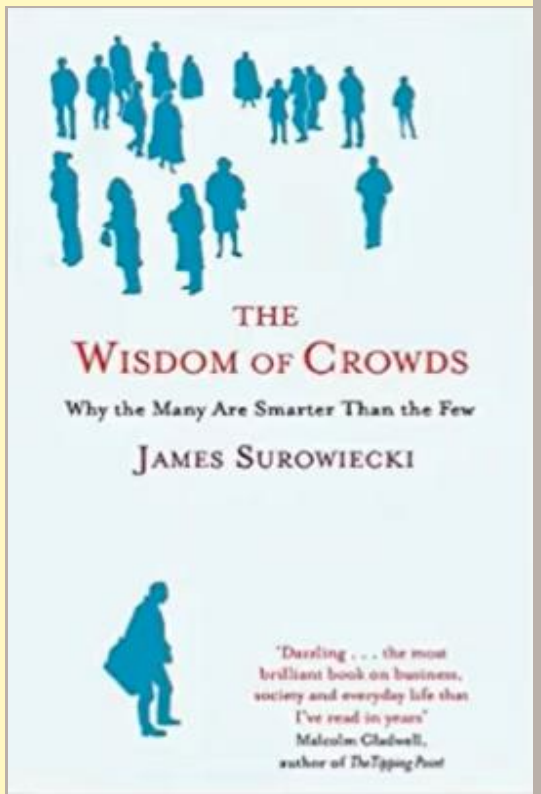
Introduction

Ensemble learning is a machine learning technique that combines multiple models (or "learners") to make predictions, aiming for better accuracy and robustness than a single model alone. It works by training several individual models and then aggregating their predictions to make a final prediction. It is similar to “wisdom of

The "**wisdom of crowds**" refers to the idea that the collective opinion of a diverse and independent group of individuals can be more accurate or insightful than that of any single expert.

For example reviews of a product. If suppose 1000 customers reviews a product 4.5 out of 5 then others are supposed to convinced to buy the same product but if only one customer reviews it then it is not likely to convince others.

Other example movie rating, democracy etc.



Ways to create Ensemble Learning

There are 3 ways

i) All the models are different with same data for training
ii) All the models are same but train with different data
iii) Combinations of i) and ii)

Types of Ensemble Learning

Generally there are 4 types of Ensemble Learning

i) Voting
ii) Bagging [Random Forest]
iii) Boosting [AdaBoost, GradiantBoost, XgBoost]
iv) Stacking

1. Voting

The voting method generally serves classification problems. In this technique, multiple models make predictions for each data point. Each model’s predictions count as a ‘vote.’ The majority of the models’ predictions determine the final prediction.

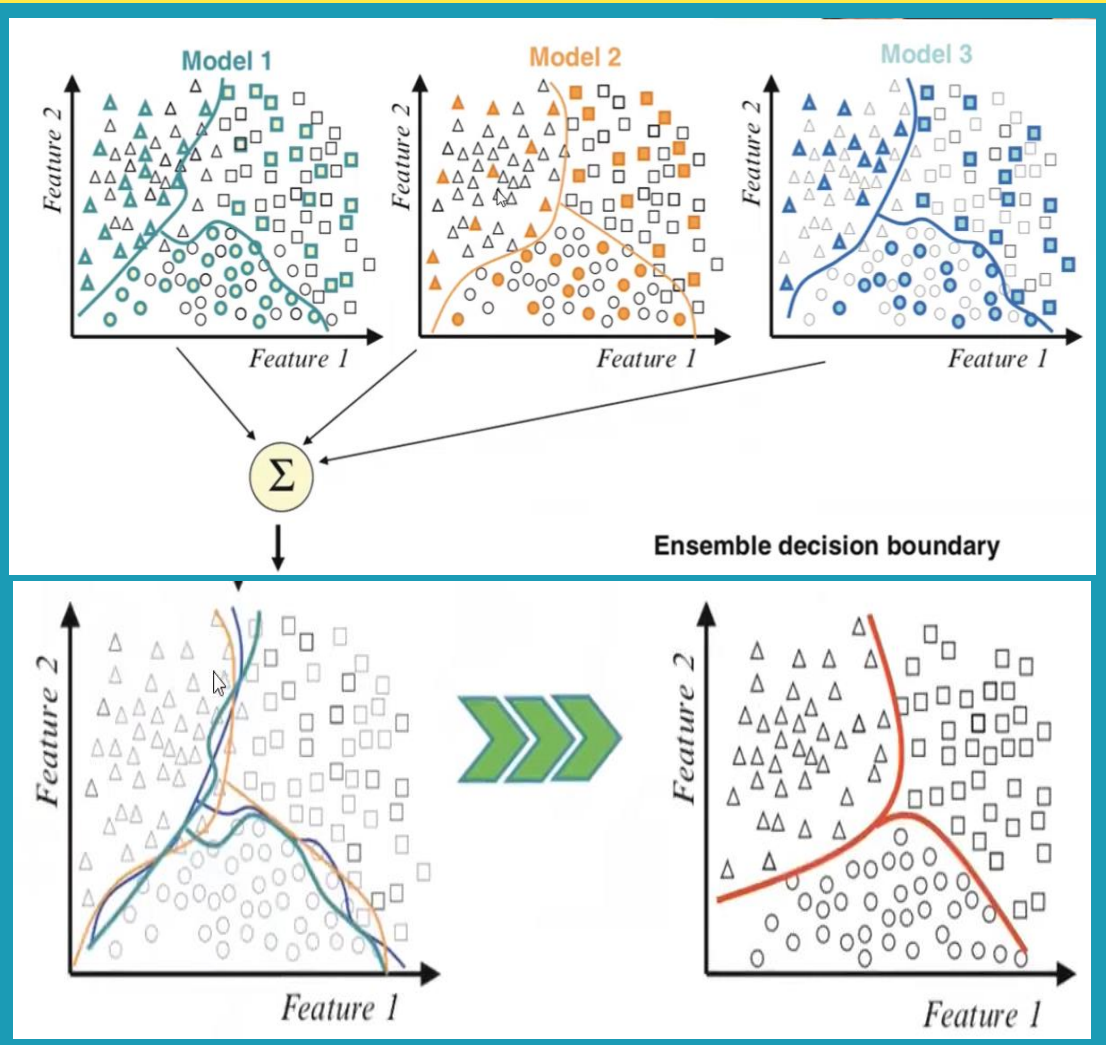
Ensemble Learning

Week-06, c-3

Why Ensemble Learning Works?

Consider the following image

Ensemble learning is a technique that combines multiple models (called **base learners**) to improve prediction accuracy and robustness. Instead of relying on a single model, ensemble methods aggregate multiple models to reduce errors, increase stability, and handle complex patterns in data.



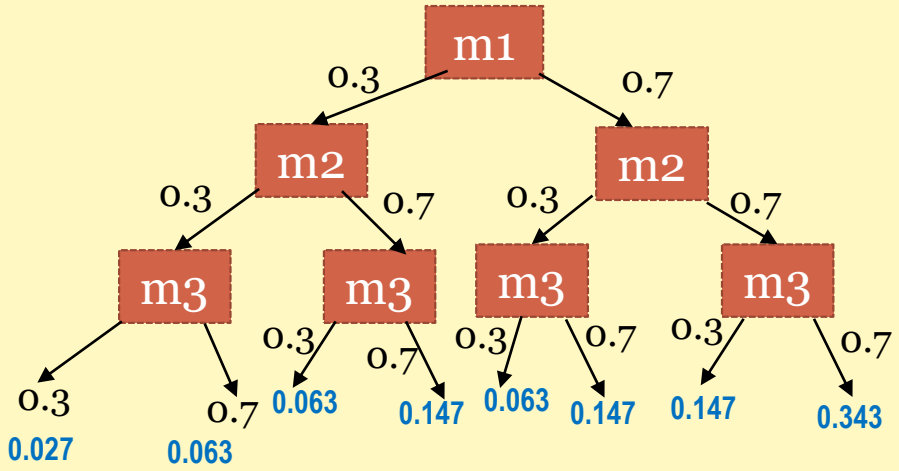
Ensemble Learning

Week-06, c-3

How Voting Improves the Performance

Suppose there are 3 same models (m1,m2 and m3) each of with accuracy of 70%.

Here calculate cumulative probability by multiplying three probabilities from three models. Assumption: accuracy of each model must be grater than 0.5.



Total Probability=0.27+3*0.063+3*0.147+0.343=1

Here we have ways where model predictions were correct

Accuracy of voting classifier=3*0.147+0.343=0.784

This accuracy clearly more than the accuracies from any of the given model.

Suppose if accuracy of each model=0.3

Accuracy of voting classifier=3*0.063+0.027=0.216

Therefore if accuracy of model less than 50% then accuracy of the voting classifier will be less than the accuracy all of the models

Program: Voting Classifier

```
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.svm import SVC
```

```
df = pd.read_csv('F:\\python\\iris.csv')
```

```
encoder = LabelEncoder()
df['variety'] = encoder.fit_transform(df['variety'])
```

```
sns.pairplot(df,hue='variety')
```

```
new_df = df[df['variety'] != 0][['sepal.length‘ ,
'sepal.width','variety']]
```

```
X = df.iloc[:,0:2]
y = df.iloc[:,-1]
```

Ensemble Learning

```
clf1 = LogisticRegression()
clf2 = RandomForestClassifier()
clf3 = KNeighborsClassifier()
```

```
estimators = [('lr',clf1),('rf',clf2),('knn',clf3)]
```

```
for estimator in estimators:
    x = cross_val_score(estimator[1],X,y,cv=10,scoring='accuracy')
    print(estimator[0],np.round(np.mean(x),2))
```

```
from sklearn.ensemble import VotingClassifier
```

```
# Hard Voting
vc = VotingClassifier(estimators=estimators,voting='hard')
x = cross_val_score(vc,X,y,cv=10,scoring='accuracy')
print(np.round(np.mean(x),2))
```

```
# Soft Voting
vc1 = VotingClassifier(estimators=estimators,voting='soft')x =
cross_val_score(vc1,X,y,cv=10,scoring='accuracy')print(np.round
(np.mean(x),2))
```

```
# Weighted Voting
for i in range(1,4):
    for j in range(1,4):
        for k in range(1,4):
            vc =
VotingClassifier(estimators=estimators,voting='soft',we
ights=[i,j,k])
            x = cross_val_score(vc,X,y,cv=10, scoring=
                'accuracy')
            print("for i={},j={},k={}".format(i,j,k)
                ,np.round(np.mean(x),2))
```

Random Forest (RF):SML

Week-11, c-3

What is Random Forest?

Random Forest is a popular machine learning algorithm that belongs to the supervised learning technique. It can be used for both Classification and Regression problems in ML. It is based on the concept of **ensemble learning,** which is a process of *combining multiple classifiers to solve a complex problem and to improve the performance of the model.*

As the name suggests, ***"Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset."*** Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output.

The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.



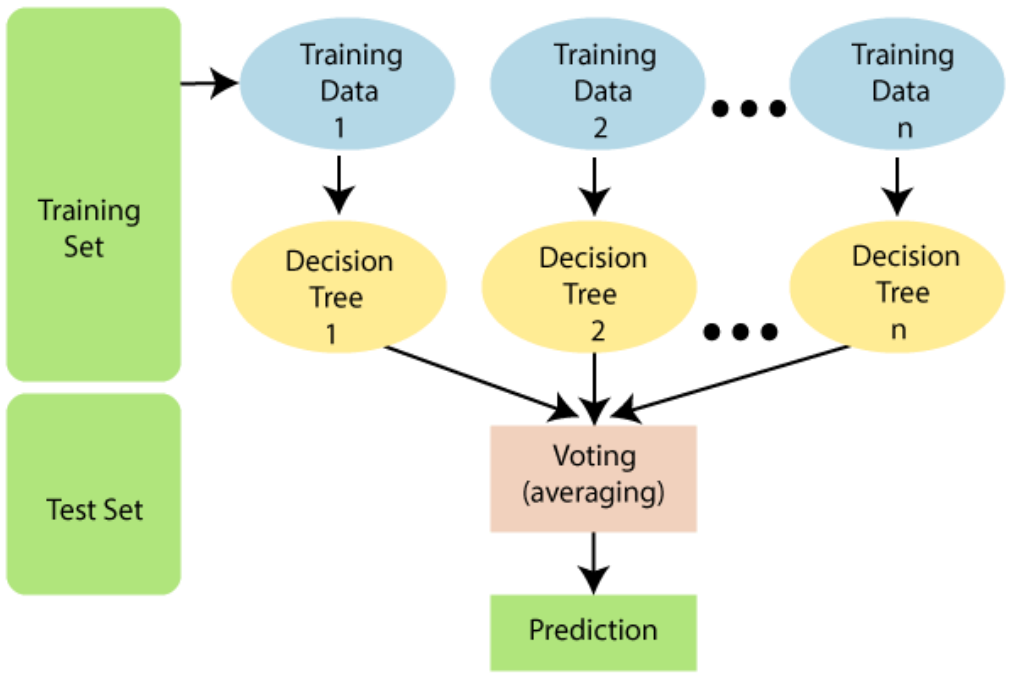
Forest: Set of Trees
Random Forest: Set of Decision Trees extracted Randomly

Random Forest (RF):SML

Week-11, C-3

Working Steps of RF

Step-1: Select random K data points from the training set.
Step-2: Build the decision trees associated with the selected data points (Subsets).
Step-3: Choose the number N for decision trees that you want to build.
Step-4: Repeat Step 1 & 2.
Step-5: For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.



Why use Random Forest?

- It takes less training time as compared to other algorithms.
- It predicts output with high accuracy, even for the large dataset it runs efficiently.
- It can also maintain accuracy when a large proportion of data is missing.

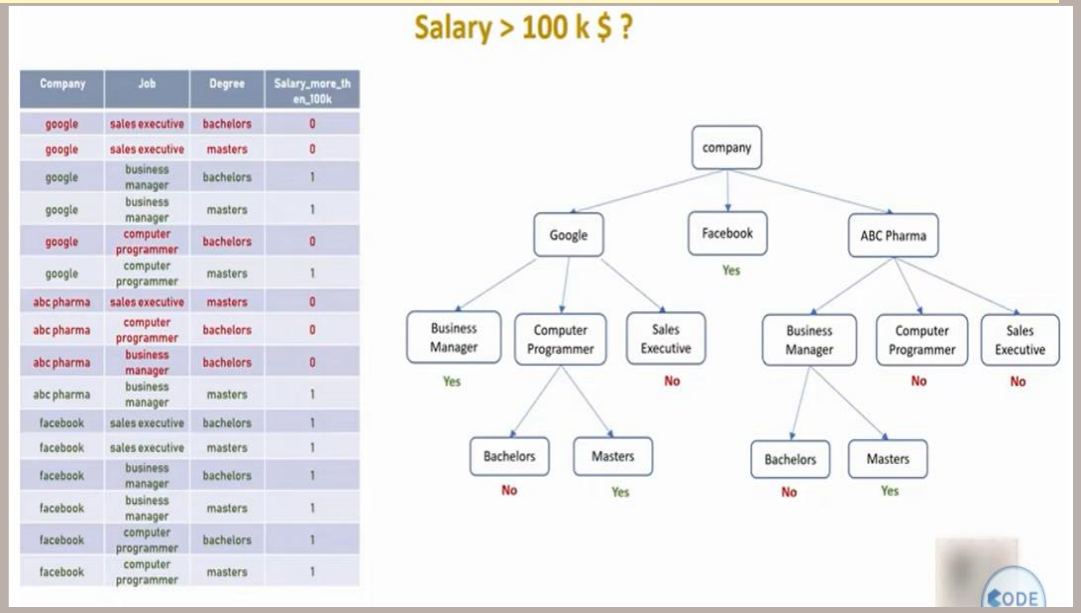
Note: To better understand the Random Forest Algorithm, you should have knowledge of the Decision Tree Algorithm.

Random Forest (RF):SML

Week-11, c-3

Example: RF

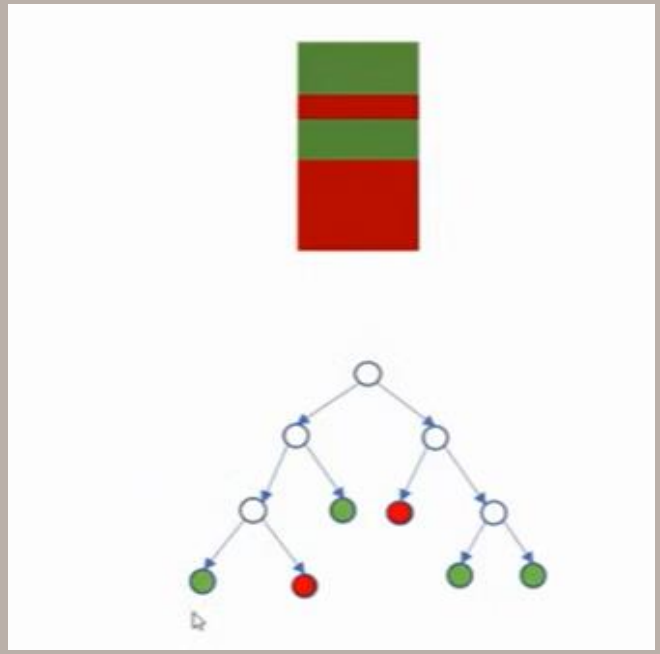
Consider the following Decision Tree



Company	Job	Degree	Salary_more_then_100k
google	sales executive	bachelors	0
google	sales executive	masters	0
google	business manager	bachelors	1
google	business manager	masters	1
google	computer programmer	bachelors	0
google	computer programmer	masters	1
abc pharma	sales executive	masters	0
abc pharma	computer programmer	bachelors	0
abc pharma	business manager	bachelors	0
abc pharma	business manager	masters	1
facebook	sales executive	bachelors	1
facebook	sales executive	masters	1
facebook	business manager	bachelors	1
facebook	business manager	masters	1
facebook	computer programmer	bachelors	1
facebook	computer programmer	masters	1

Sometimes it becomes very complex diagram

Simplified RF



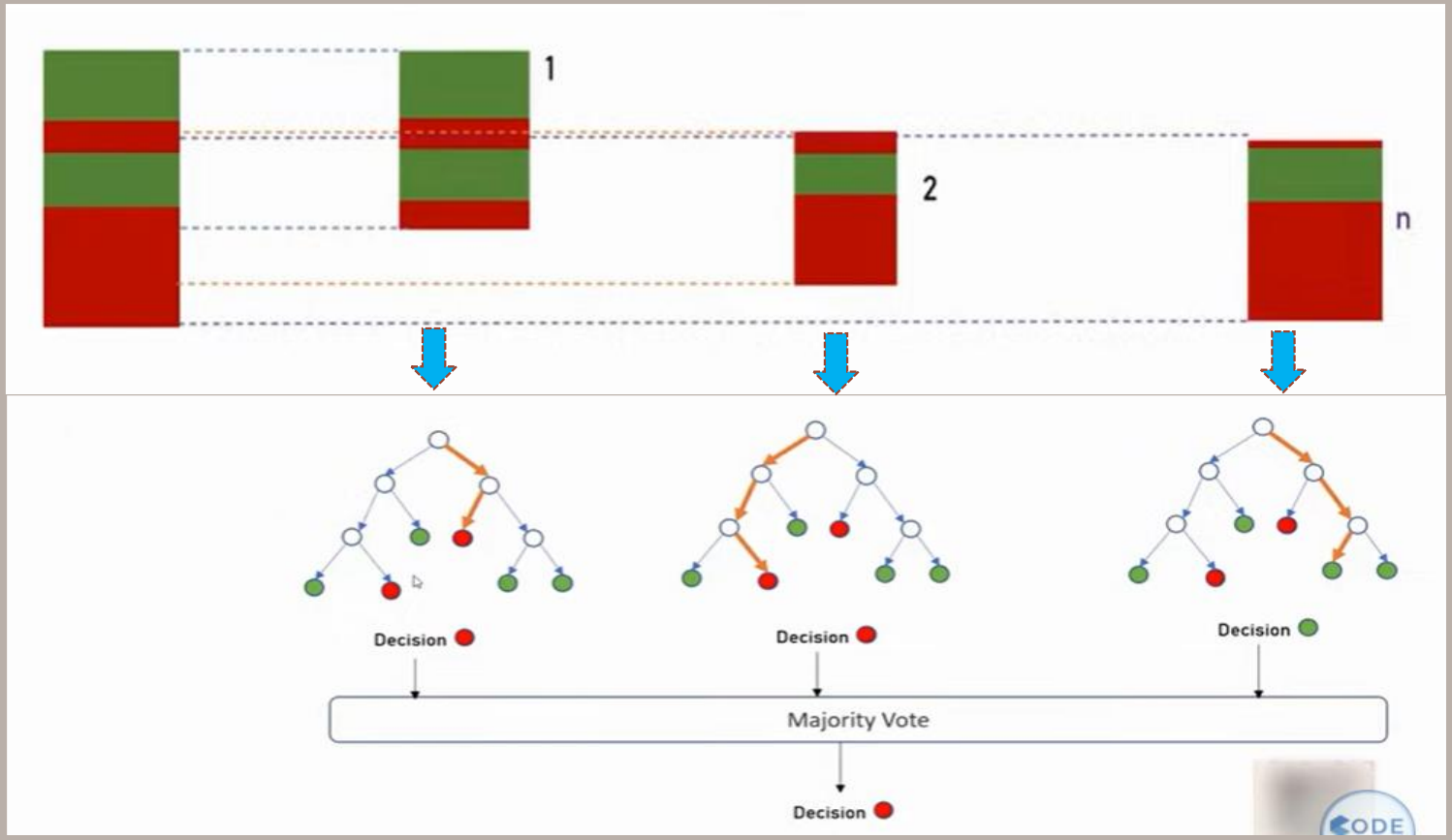
Green and Red Color denotes classes

Random Forest (RF):SML

Week-11, c-3

Example: RF

Device the Dataset randomly to create trees

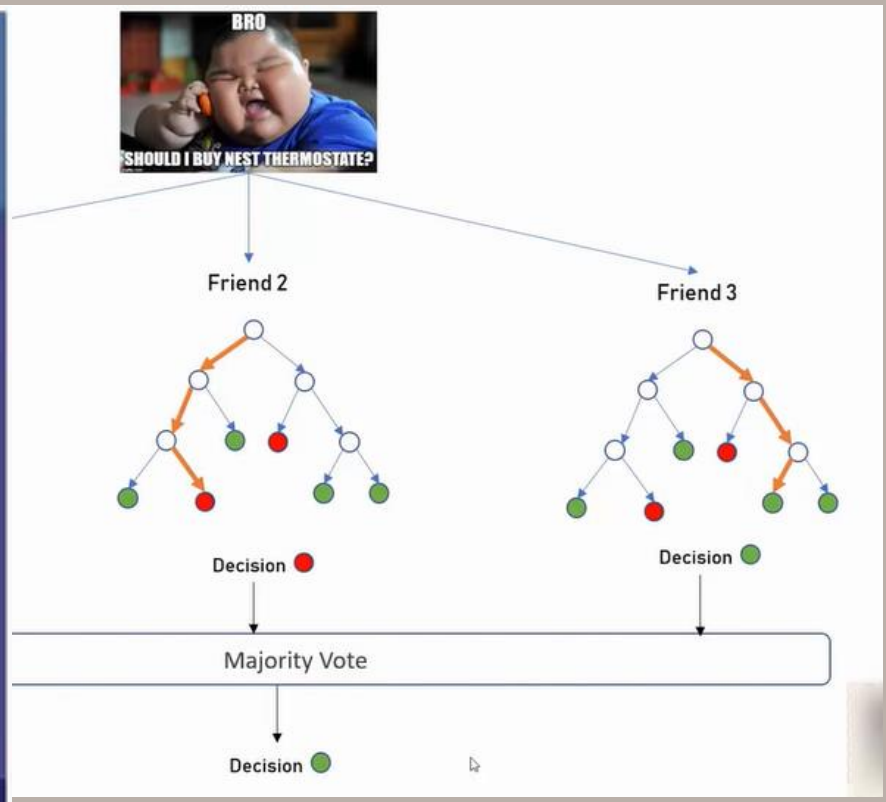


Then train each tree. Get prediction from each tree and finalize the prediction by voting.

Random Forest (RF):SML

Week-11, c-3





ds whether he buy biryani from Sultan
s him their opinions. Based on their
cide for buying biryani from sultan.

Topics: RF

Week-11

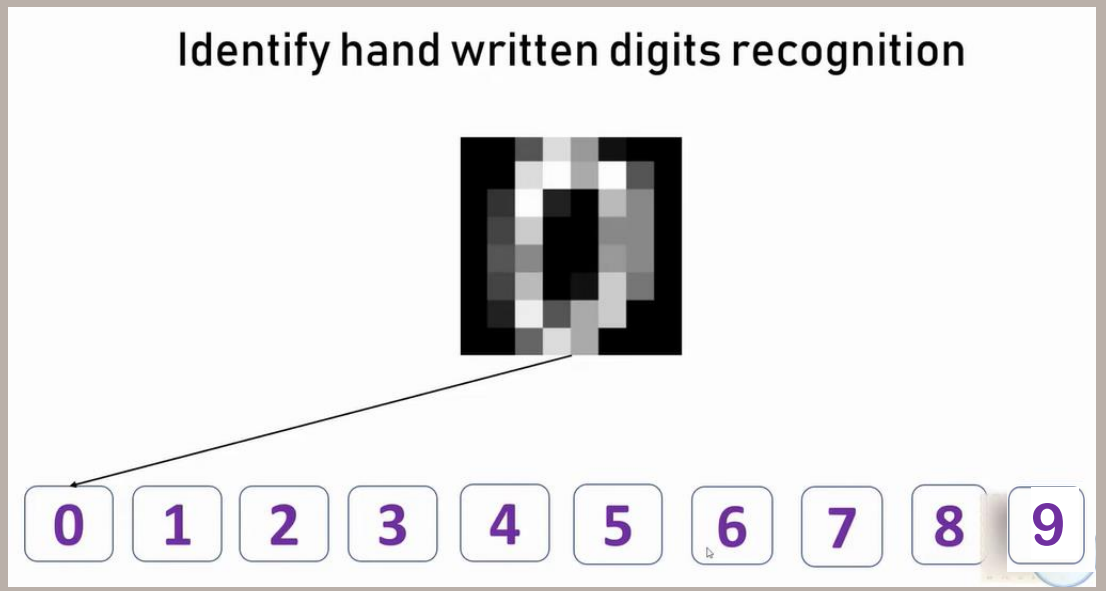
Python Example

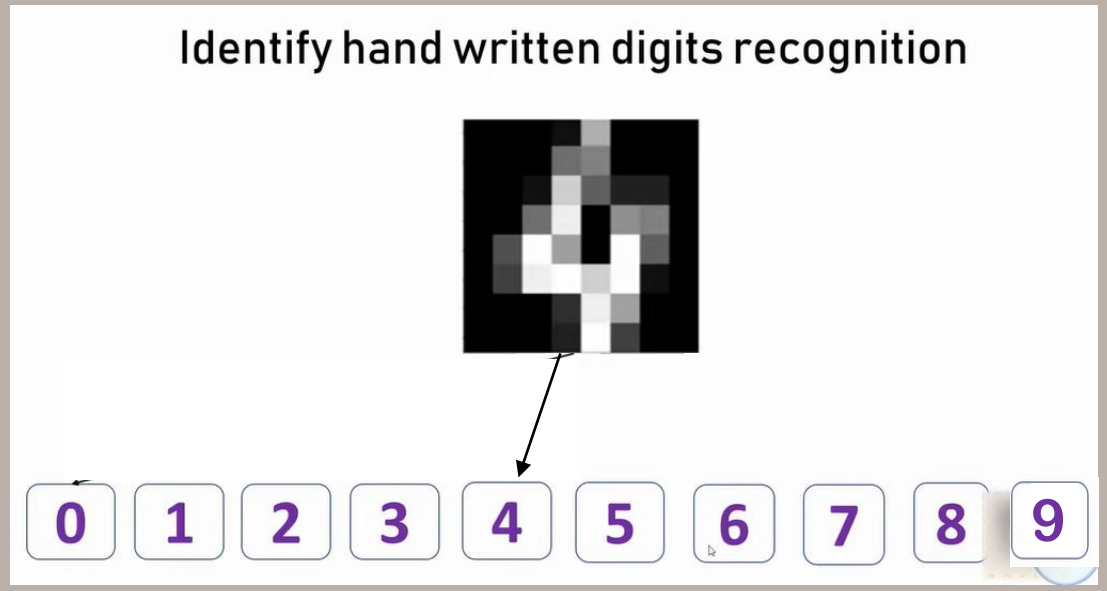
DT (Python Example)

Week-08, c-3

Example: Sklearn Digit Dataset

This dataset is made up of 1797 8x8 images. Each image, like the one shown below, is of a hand-written digit. In order to utilize an 8x8 figure like this, we'd have to first transform it into a feature vector with length 64.





Predicts digits 0 to 9

DT (Python Example)

Week-08, c-3

Example: Sklearn Digit Dataset

```
import pandas as pd
from sklearn.datasets import load_digits
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix,
classification_report
digits=load_digits()
```

```
plt.gray()
for i in range(4):
    plt.matshow(digits.images[i])
```

```
df=pd.DataFrame(digits.data)
```

```
df['target']=digits.target
```

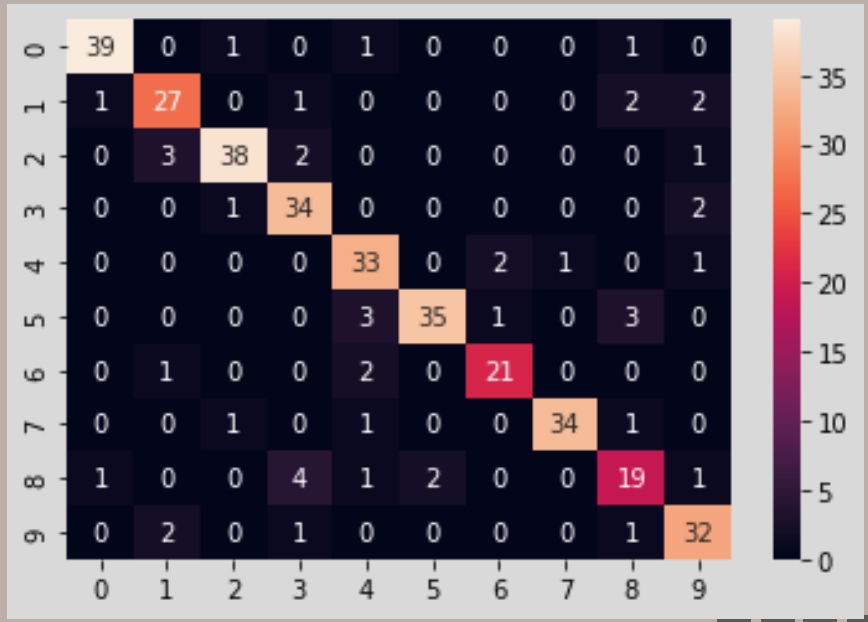
```
X=df.drop(['target'],axis='columns')
y=df.target
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2)
```

```
dt=DecisionTreeClassifier()
dt.fit(X_train,y_train)
dt.score(X_test,y_test)
```

```
rf=RandomForestClassifier()
rf.fit(X_train,y_train)
rf.score(X_test,y_test)
```

```
y_pred_dt=dt.predict(X_test)
cm=confusion_matrix(y_test,y_pred_dt)
sns.heatmap(cm, annot=True)
```

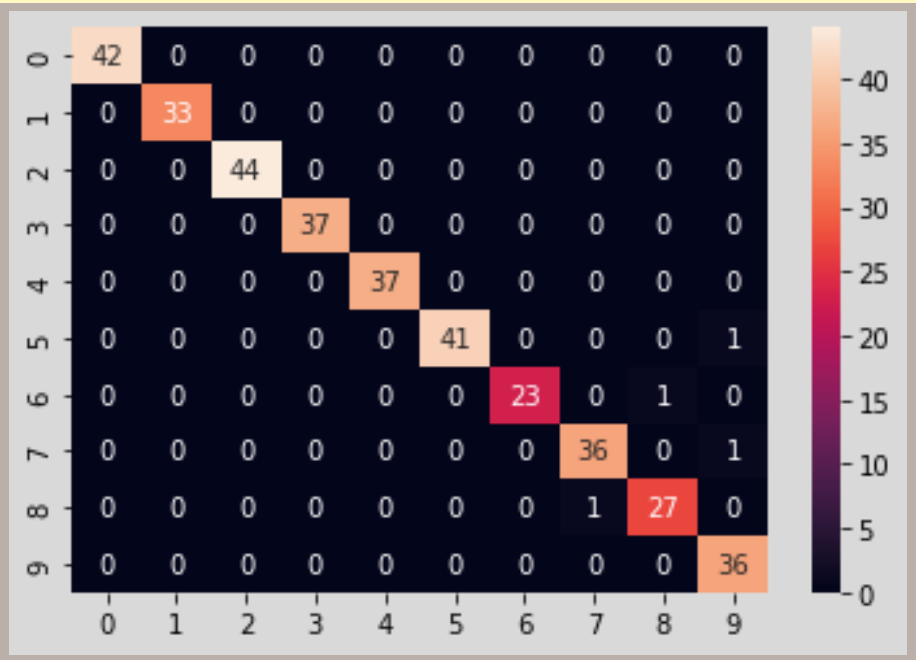


DT (Python Example)

Week-08, c-3

Example: Sklearn Digit Dataset

```
y_pred_rf=rf.predict(X_test)
cm=confusion_matrix(y_test,y_pred_rf)
sns.heatmap(cm, annot=True)
```

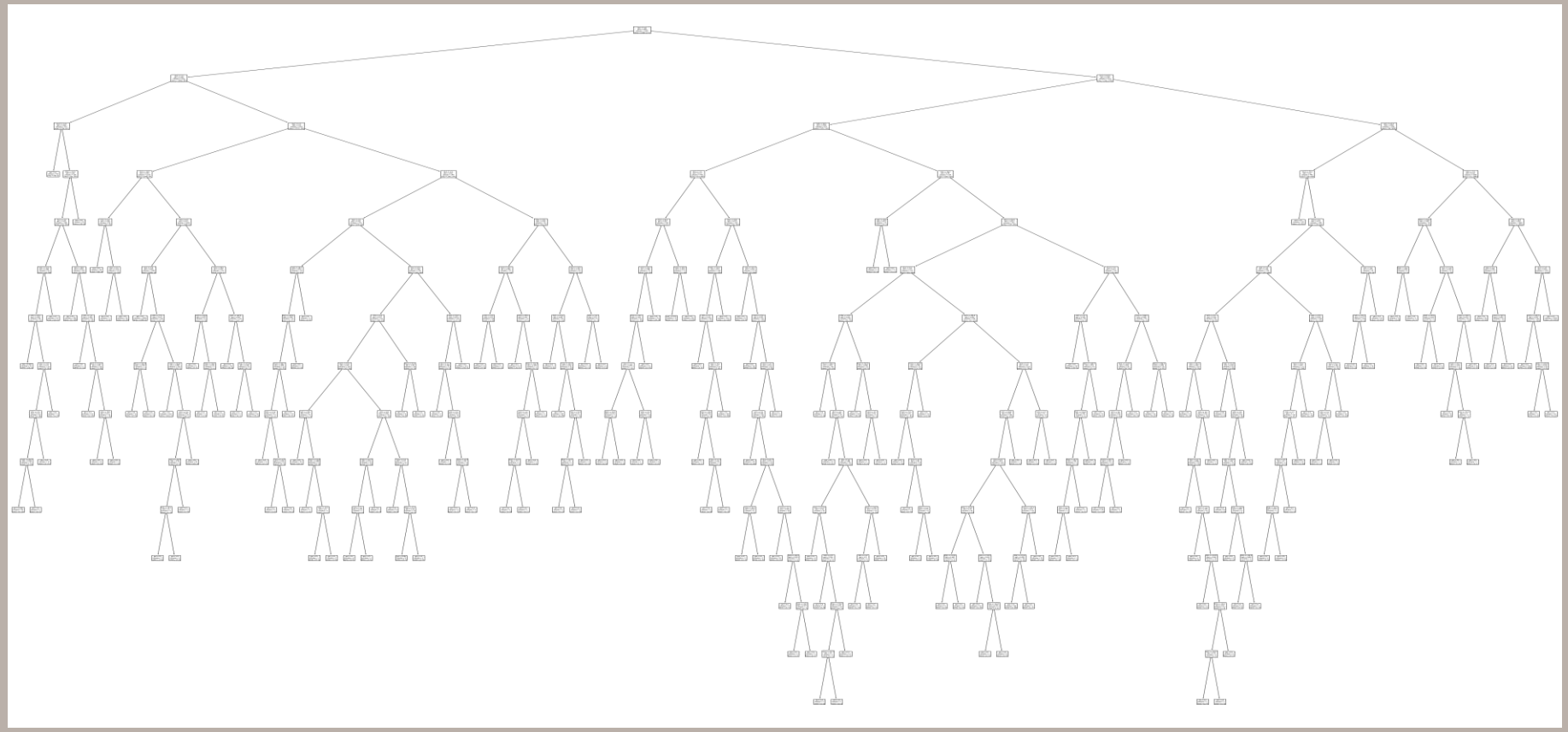


DT (Python Example)

Week-08, c-3

Example: Saray Dataset

```
from sklearn.tree import plot_tree
from matplotlib.pylab import rcParams
rcParams['figure.figsize']=80,50
plot_tree(model)
```



DT (Python Example)

Week-08, c-3

Example: Iris Dataset

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix,
classification_report
```

```
iris=load_iris()
X=pd.DataFrame(iris.data,columns=iris.feature_names)
y=iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=1)
```

```
dt=DecisionTreeClassifier()
dt.fit(X_train,y_train)
```

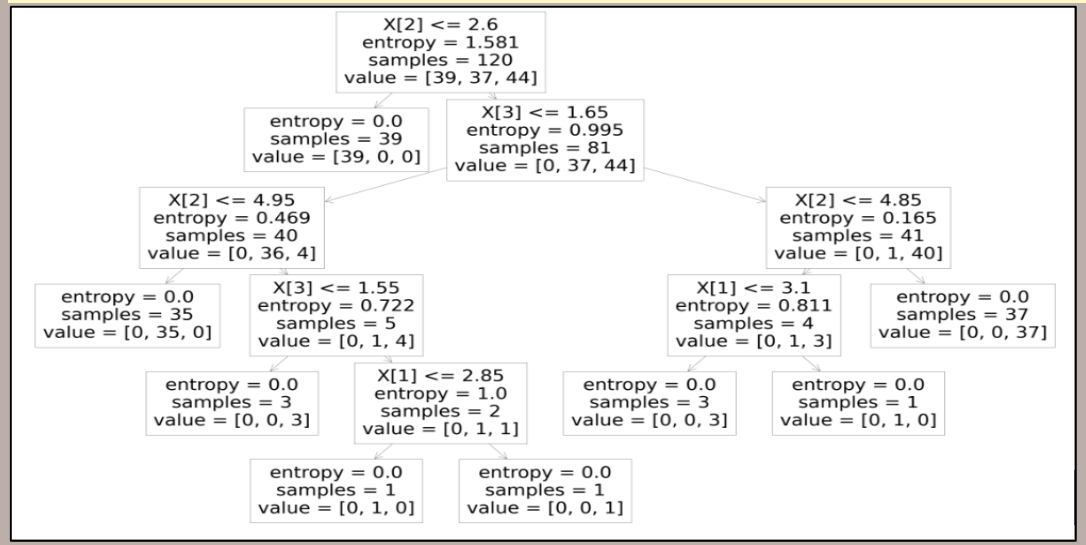
```
y_pred=dt.predict(X_test)
cm=confusion_matrix(y_test,y_pred)
cm
```

```
array([[11,  0,  0],
       [ 0, 12,  1],
       [ 0,  0,  6]], dtype=int64)
```

```
print(classification_report(y_test,y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	1.00	0.92	0.96	13
2	0.86	1.00	0.92	6
accuracy			0.97	30

```
from sklearn.tree import plot_tree
from matplotlib.pylab import rcParams
rcParams['figure.figsize']=80,50
plot_tree(dt)
```



Week-08

Overfitting & Underfitting

Overfitting & Underfitting

Week-08, c-3

Overfitting

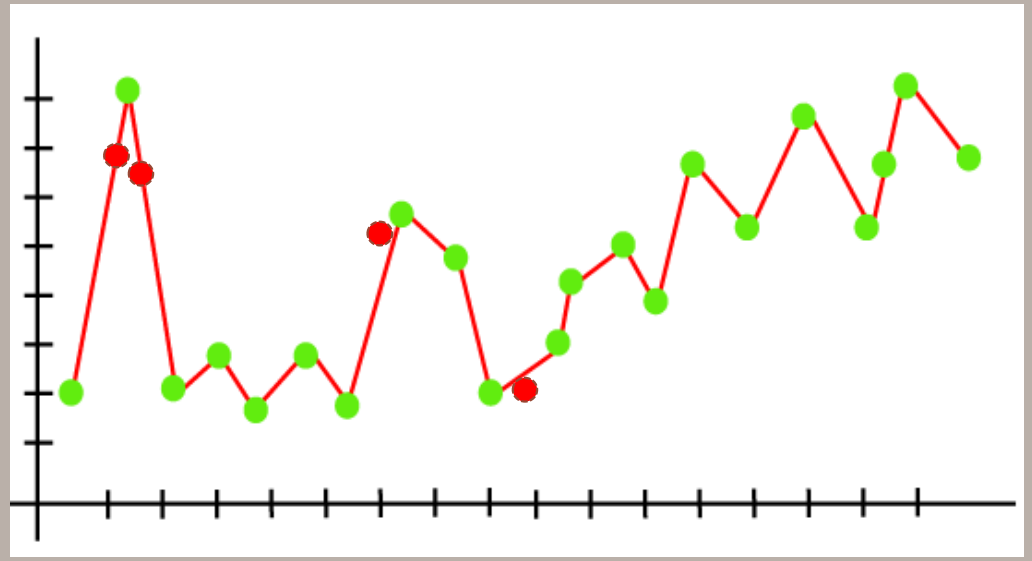
Overfitting occurs when our machine learning model tries to cover all the data points or more than the required data points present in the given dataset. Because of this, the model starts caching noise and inaccurate values present in the dataset, and all these factors reduce the efficiency and accuracy of the model. The overfitted model has low bias and high variance.

The chances of occurrence of overfitting increase as much we provide training to our model. It means the more we train our model, the more chances of occurring the overfitted model.

Overfitting is the main problem that occurs in supervised learning.

Exmaple: Overfitting

The concept of the overfitting can be understood by the below graph of the linear regression output:
Suppose here x-axis and y-axis denotes number of hours of study and GPA obtained by students



The model works very well for training but not well for testing. Ex: look at the prediction for red point, they are not good predictions.
As we can see from the above graph, the model tries to cover all the data points present in the scatter plot. It may look efficient, but in reality, it is not so. Because the goal of the regression model to find the best fit line, but here we have not got any best fit, so, it will generate the prediction errors.

Overfitting & Underfitting

Week-08, c-3

How to avoid the Overfitting

Both overfitting and underfitting cause the degraded performance of the machine learning model. But the main cause is overfitting, so there are some ways by which we can reduce the occurrence of overfitting in our model.

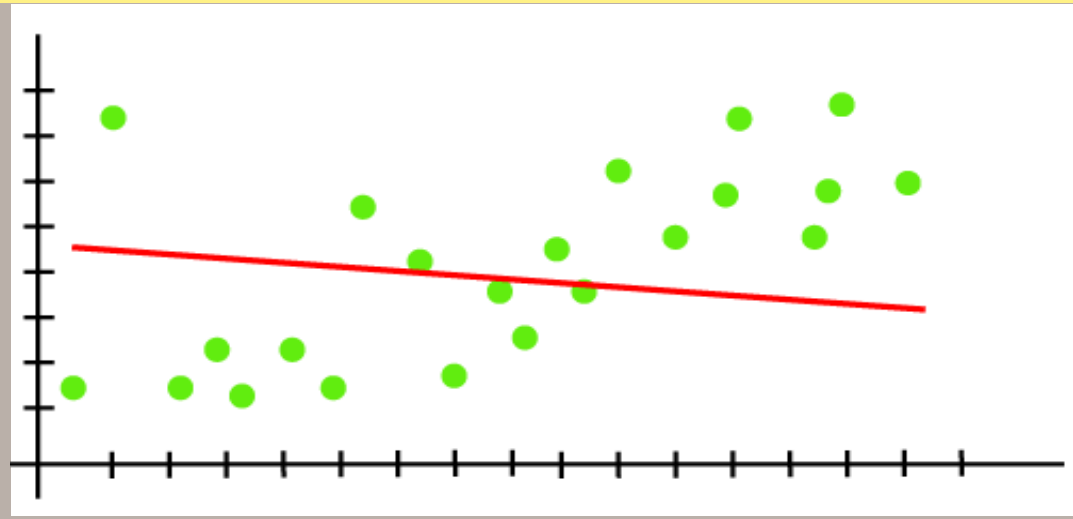
- Cross-Validation
- Training with more data
- Removing features
- Early stopping the training
- Regularization
- Ensembling

Underfitting

Underfitting occurs when our machine learning model is not able to capture the underlying trend of the data. To avoid the overfitting in the model, the fed of training data can be stopped at an early stage, due to which the model may not learn enough from the training data. As a result, it may fail to find the best fit of the dominant trend in the data.
In the case of underfitting, the model is not able to learn enough from the training data, and hence it reduces the accuracy and produces unreliable predictions.
An underfitted model has high bias and low variance.

Example: Underfitting

We can understand the underfitting using below output of the linear regression model:



As we can see from the above diagram, the model is unable to capture the data points present in the plot.

How to avoid underfitting:

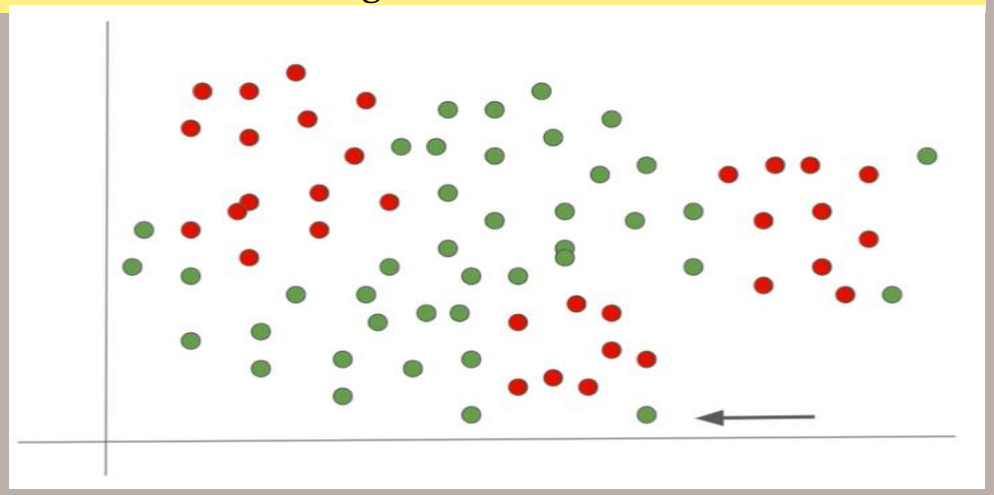
- By increasing the training time of the model.
- By increasing the number of features.

Overfitting & Underfitting

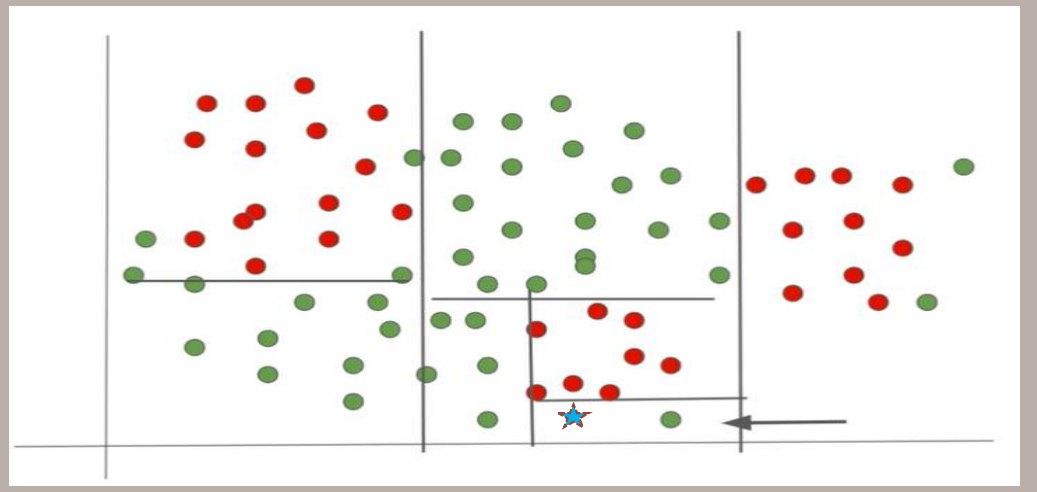
Week-08, c-3

Overfitting in DT

Consider the following dataset



Example: After few split



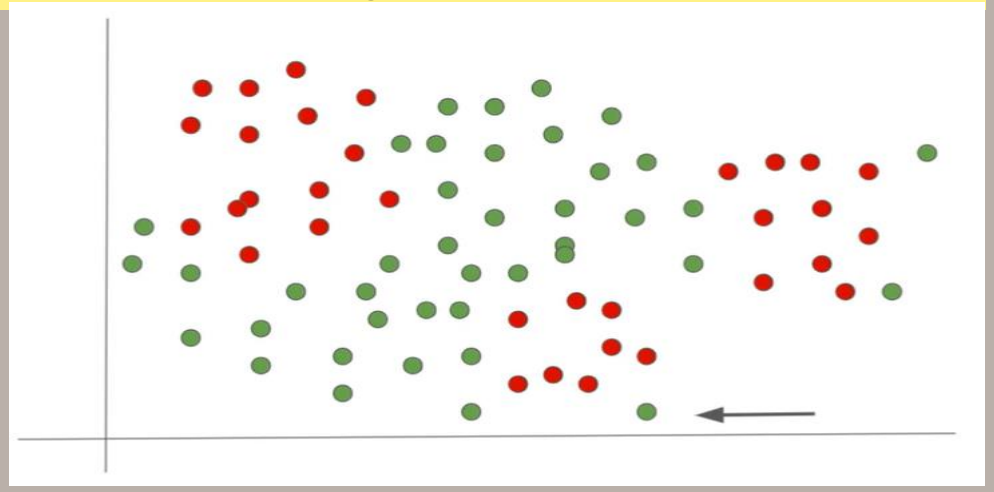
Consider the point pointed by an arrow is only one point in the box, so all points inside the box classified as green level although many points are very near to red level like blue '*' point. Although blue '*' should classified as red level but due to overfitting it will be classified as green level.

Overfitting & Underfitting

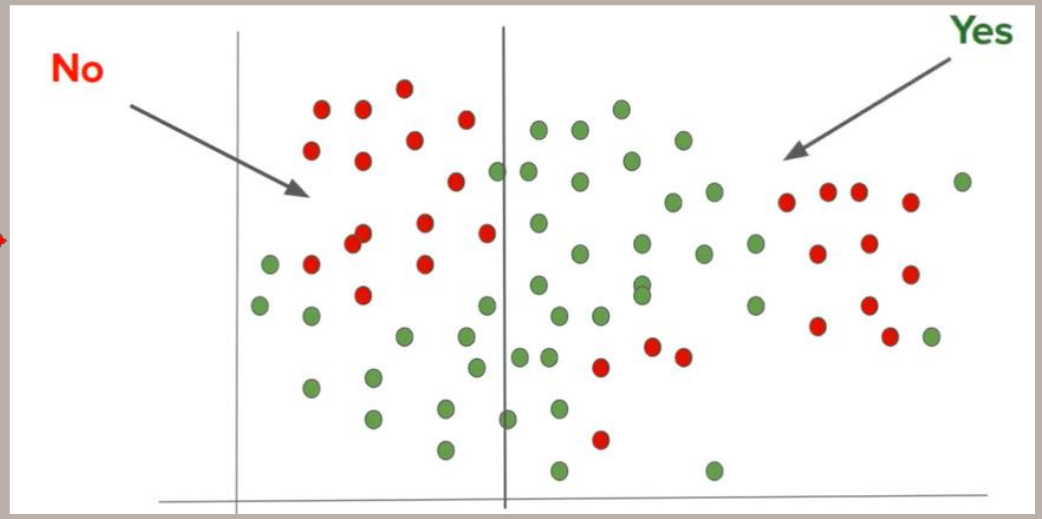
Week-08, c-3

Underfitting in DT

Consider the following dataset



Example: one split

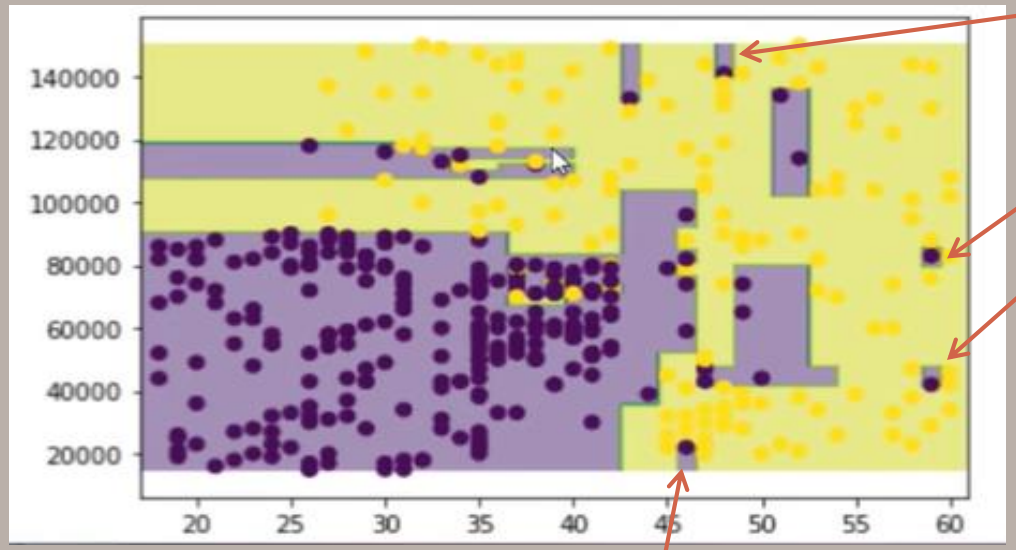


Here Depth=1 (only one split)
So left side is predicted as No and right side predicted as Yes but it is somewhat incorrect.

Overfitting & Underfitting

Week-08, c-3

Example: Overfitting in DT

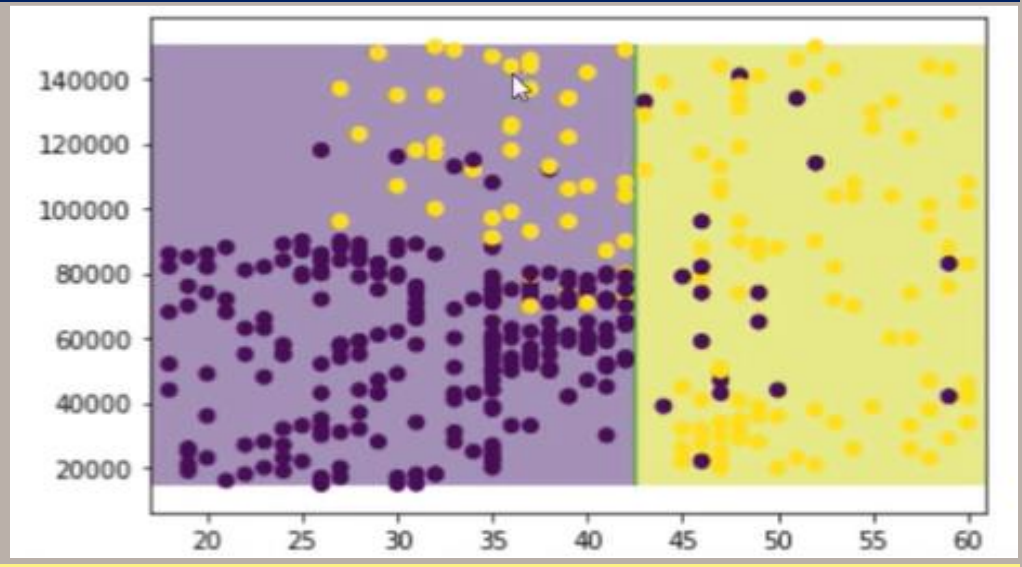


This point should be in yellow class

This point should be in yellow class

To minimize overfitting & underfittin
We need to optimize the depth of split

Example: Underfitting in DT



There are so many yellow points in left side and vice -versa

Overfitting & Underfitting

Week-08, c-3

Python Example

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score,
confusion_matrix, classification_report
from sklearn.tree import DecisionTreeClassifier
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df=pd.read_csv("F:\\python\\Juicequality.csv")
df.isnull().sum()
```

```
df.quality.value_counts()
```

```
quality_mapping={3:0,4:1,5:2,6:3,7:4,8:5}
df.loc[:,"quality"]=df.quality.map(quality_mapping)
df.quality.value_counts()
```

```
X=df.drop("quality",axis='columns')
y=df.quality
X_train,X_test,y_train,y_test=train_test_split(X,y,test_
size=0.2)
```

```
dt=DecisionTreeClassifier(max_depth=1)
dt.fit(X_train,y_train)
```

```
y_pred_test=dt.predict(X_test)
y_pred_train=dt.predict(X_train)
```

```
print(accuracy_score(y_train,y_pred_train))
```

```
0.512118428459734
```

```
print(accuracy_score(y_test,y_pred_test))
```

```
0.503125
```

```
from sklearn import tree
%matplotlib inline
train_accuracies=[]
test_accuracies=[]
for depth in range(1,25):
    tree_model=DecisionTreeClassifier(max_depth=depth)
    tree_model.fit(X_train,y_train)
    train_pred=tree_model.predict(X_train)
    test_pred=tree_model.predict(X_test)
    train_accuracy=accuracy_score(y_train,train_pred)
    test_accuracy=accuracy_score(y_test,test_pred)

    train_accuracies.append(train_accuracy)
    test_accuracies.append(test_accuracy)
```

Overfitting & Underfitting

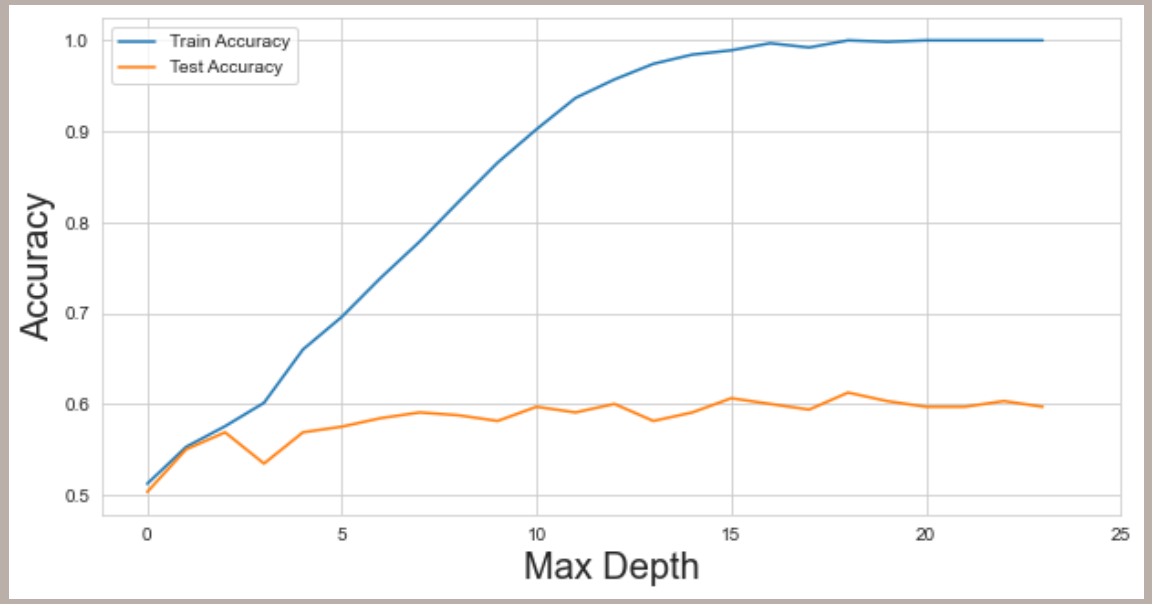
Week-08, c-3

Python Example

```
from sklearn import tree
%matplotlib inline
train_accuracies=[]
test_accuracies=[]
for depth in range(1,25):
    tree_model=DecisionTreeClassifier(max_depth=depth)
    tree_model.fit(X_train,y_train)
    train_pred=tree_model.predict(X_train)
    test_pred=tree_model.predict(X_test)
    train_accuracy=accuracy_score(y_train,train_pred)
    test_accuracy=accuracy_score(y_test,test_pred)

    train_accuracies.append(train_accuracy)
    test_accuracies.append(test_accuracy)
```

```
plt.figure(figsize=(10,5))
sns.set_style("whitegrid")
plt.plot(train_accuracies,label="Train Accuracy")
plt.plot(test_accuracies,label="Test Accuracy")
plt.xticks(range(0,26,5))
plt.legend(loc="upper left")
plt.xlabel("Max Depth",size=20)
plt.ylabel("Accuracy",size=20)
```



We can see clearly that train accuracy is high but test accuracy is low. It is overfitting. The training accuracy is increasing with the depth of the tree.

End of Chapter